Feed-Forward Networks, Normalization, and Residual Connections

pig7selene

September 5, 2026

Abstract

Attention moves information across token positions, but it is not the only source of computation in a Transformer block. This chapter develops the position-wise Feed-Forward Network as a learned expansion, nonlinear transformation, and contraction; derives gated variants and SwiGLU; and compares LayerNorm with RMSNorm at the level of their statistics and axes. It then treats residual connections, normalization placement, initialization, and gradient flow as one coupled design problem, concluding with parameter, compute, activation, and numerical implementation contracts relevant to modern decoder-only models.

1 Introduction

A decoder-only Transformer block alternates two qualitatively different operations. Self-Attention mixes information across positions, whereas the Feed-Forward Network (FFN) transforms the features of each position independently. Normalization determines the scale at which each sublayer reads the residual stream, and the residual connection determines how the resulting update is accumulated. These components are sometimes presented as architectural details surrounding attention. In a deep language model they are instead part of the mechanism that makes repeated sequence mixing expressive and trainable.

Let $H \in R^{B\times T\times d}$ denote a residual-stream tensor with batch size B, sequence length T, and model width d. The FFN intermediate width is denoted by m. Every operation considered here preserves the outer shape $B \times T$; normalization and the FFN act independently on the final feature axis, while residual addition requires the sublayer output to return to width d.

2 Position-Wise Feed-Forward Networks

2.1 Expansion, Nonlinearity, and Contraction

The original Transformer applies the same two-layer network to every sequence position [1]. For a single feature vector $x \in R^d$, a conventional FFN is

$$\mathrm{FFN}(x) = \varphi\big(xW_{\mathrm{up}} + b_{\mathrm{up}}\big)W_{\mathrm{down}} + b_{\mathrm{down}}, \tag{1}$$

with $W_{\mathrm{up}} \in R^{d\times m}$, $b_{\mathrm{up}} \in R^m$, $W_{\mathrm{down}} \in R^{m\times d}$, and $b_{\mathrm{down}} \in R^d$. Applied to H, the first affine map produces a tensor in $R^{B\times T\times m}$; the second returns it to $R^{B\times T\times d}$. The weights are shared across all positions and batch items. Consequently, Equation 1 mixes features but never token positions.

The width m is usually larger than d. Expansion gives the nonlinearity a higher-dimensional feature space in which to form an update; contraction makes that update compatible with the residual stream. Without the nonlinear function φ, the two affine maps would collapse into a single affine transformation, apart from their combined bias, and the expanded representation would add no expressive depth. Ignoring biases, Equation 1 contains $2dm$ parameters. For each token it also requires approximately $2dm$ scalar multiply-accumulates: dm for each projection. If one multiplication and one addition are counted as two floating-point operations, this is approximately $4dm$ FLOPs. Stating the counting convention matters because systems reports use both conventions.

2.2 ReLU, GELU, and SiLU

The original Transformer used the Rectified Linear Unit,

$$\mathrm{ReLU}(z) = \max(0, z), \tag{2}$$

applied elementwise. Later language models commonly use smoother functions. The Gaussian Error Linear Unit is

$$\mathrm{GELU}(z) = z\Phi(z), \tag{3}$$

where Φ is the standard normal cumulative distribution function [2]. The Sigmoid Linear Unit, usually implemented as SiLU or Swish, is

$$\mathrm{SiLU}(z) = z\sigma(z), \tag{4}$$

where $\sigma(z) = \frac{1}{1+\exp(-z)}$. ReLU removes every negative input exactly. GELU and SiLU instead attenuate negative inputs smoothly and retain a small nonzero response over part of the negative half-line. The choice changes optimization and representation behavior, but its effect cannot be reduced to smoothness alone; model scale, initialization, precision, and the surrounding gated architecture also matter.

3 Gated Feed-Forward Networks and SwiGLU

3.1 Multiplicative Gating

A gated FFN replaces the single transformed branch in Equation 1 with the elementwise product of two learned projections. In a bias-free form,

$$\mathrm{GFFN}(x) = \big(\varphi(xW_g) \odot (xW_u)\big)W_d, \tag{5}$$

where $W_g, W_u \in R^{d\times m}$, $W_d \in R^{m\times d}$, and $\odot$ denotes the Hadamard product. One branch determines a data-dependent gate and the other supplies candidate features. Because both branches depend on x, the product introduces a multiplicative interaction that is absent from a single activation applied after one projection.

SwiGLU chooses SiLU for the gated branch:

$$\mathrm{SwiGLU}(x) = \big(\mathrm{SiLU}(xW_g) \odot (xW_u)\big)W_d. \tag{6}$$

Shazeer compared several GLU variants in Transformer FFNs and introduced the SwiGLU naming and formulation [3]. Modern decoder-only models often combine SwiGLU with Pre-Norm and RMSNorm; LLaMA is a prominent documented example [4]. This convention is empirical rather than a mathematical requirement: the gate, normalization, residual organization, and intermediate width remain separable architectural choices.

3.2 Parameter-Matched Intermediate Width

The extra projection changes the appropriate comparison of widths. A conventional FFN of width m_v has approximately $2dm_v$ weights, whereas a gated FFN of width m_g has approximately $3dm_g$. Matching their leading parameter counts gives

$$2dm_v \approx 3dm_g \quad \Rightarrow \quad m_g \approx \frac{2}{3}m_v. \tag{7}$$

Thus a vanilla expansion $m_v = 4d$ corresponds to a gated width near $8\frac{d}{3}$ at the same leading parameter budget. Implementations usually round m_g to a hardware-friendly multiple. Comparing a $4d$ vanilla FFN directly with a $4d$ SwiGLU block silently gives the gated model roughly fifty percent more projection parameters and multiply-accumulates.

4 Normalization over the Feature Axis

Normalization in a Transformer is applied independently to each token vector. It does not aggregate statistics across the batch or across sequence positions. Let $x \in R^d$, let $\varepsilon > 0$ be a numerical stabilizer, and let $\gamma, \beta \in R^d$ be learned elementwise affine parameters where present.

4.1 LayerNorm

LayerNorm computes the feature mean and population variance

$$\mu(x) = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2(x) = \frac{1}{d}\sum_{i=1}^{d} (x_i - \mu(x))^2, \tag{8}$$

then returns

$$\mathrm{LayerNorm}\,(x)_i = \gamma_i \frac{x_i - \mu(x)}{\sqrt{\sigma^2(x) + \varepsilon}} + \beta_i. \tag{9}$$

The mean and variance are recomputed for each token at both training and inference time; they do not depend on batch-level running statistics [5]. LayerNorm is invariant to a uniform shift of all features before the learned affine transformation and is also invariant to positive rescaling, up to the effect of ε.

4.2 RMSNorm

RMSNorm removes mean subtraction and normalizes by the root mean square,

$$\mathrm{rms}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}, \tag{10}$$

$$\mathrm{RMSNorm}\,(x)_i = \gamma_i \frac{x_i}{\mathrm{rms}(x)}. \tag{11}$$

In its usual form there is no learned bias β. RMSNorm therefore uses the second raw moment rather than the variance: it provides rescaling invariance but not recentering invariance. Zhang and Sennrich proposed RMSNorm as a computationally simpler alternative to LayerNorm and found recentering dispensable in their studied settings [6]. That empirical result does not make the two transformations identical; a feature vector with a large nonzero mean is treated differently by Equation 9 and Equation 11.

Method	Statistic	Core transform	Typical affine parameters
LayerNorm	Mean and centered variance	Center, then divide by standard deviation	γ, β
RMSNorm	Root second moment	Divide by RMS without centering	γ

Table 1: LayerNorm and RMSNorm operate over the feature axis of each token vector. The table describes their standard forms; implementation variants may alter the affine parameters.

4.3 Numerical Implementation

Both normalizers reduce d values into one scale, so their implementation must balance bandwidth, precision, and kernel fusion. The stabilizer ε belongs inside the square root in Equation 9 and Equation 10. In mixed-precision training, it is common to accumulate the reduction in higher precision than the stored activations and then cast the normalized result as required by the surrounding kernel. A mathematically equivalent rearrangement is not automatically numerically equivalent: computing the variance as $E[x^2] - E[x]^2$, for example, can suffer cancellation when the mean is large.

The affine parameters are broadcast over the B and T axes. A frequent implementation error is to normalize over more than the last feature axis, which couples tokens or batch items and changes the model. Another is to insert a second, unintended ε or to use different values across training and inference. These choices are part of the checkpoint's executable specification.

5 Residual Connections and Block Ordering

5.1 The Residual Stream

Let F_l denote the learned transformation of sublayer l, including either attention or an FFN. A residual update has the shape contract

$$H^l = H^{l-1} + \Delta H^l, \quad H^{l-1}, \Delta H^l \in R^{B\times T\times d}. \tag{12}$$

The tensor H is called the residual stream because every sublayer reads from and writes an update to a common width-d representation. The additive path can preserve existing information while the learned branch contributes a correction. It also imposes a strict interface: an FFN may expand internally to m, but its contraction must return to d before residual addition.

5.2 Post-Norm and Pre-Norm

The original Transformer used Post-Norm sublayers [1],

$$y = N(x + F(x)), \tag{13}$$

where normalization follows residual addition. A Pre-Norm sublayer instead computes

$$y = x + F(N(x)). \tag{14}$$

The two forms are not interchangeable. Pre-Norm presents a normalized input to F while leaving an exact identity path from x to y. Post-Norm normalizes the combined stream, so the identity branch also passes through the Jacobian of N. GPT-2 documented moving normalization to the input of each sub-block and adding a final normalization after the stack [7]; Pre-Norm families have since become common in decoder-only models.

For a Pre-Norm stack, repeated substitution exposes the additive structure

$$H^L = H^0 + \sum_{l=1}^{L} F_{l\left(N_{l(H^{l-1})}\right)}. \tag{15}$$

This equation does not imply that layers are independent: each update depends on the accumulated stream. It does show that a forward identity route persists through arbitrary depth, and the backward Jacobian of one sublayer has the form

$$\frac{\partial y}{\partial x} = I + J_F J_N. \tag{16}$$

For Post-Norm, the corresponding local Jacobian is $J_{N(I+J_F)}$. The placement of J_N changes how gradients compose across depth, which is one reason normalization location affects optimization rather than merely activation scale.

6 Initialization, Gradient Flow, and Stability

6.1 Variance-Preserving Scale

Initialization should keep activations and gradients in a useful numerical range before learning has organized the network. Consider a bias-free projection $y = xW$ with $W \in R^{d_{\text{in}}\times d_{\text{out}}}$. If the components of x and W are independent, zero mean, and have variances q and s^2, then

$$\operatorname{Var}(y_j) \approx d_{\text{in}} s^2 q. \tag{17}$$

Choosing s^2 on the order of $\frac{1}{d_{\text{in}}}$ therefore preserves variance to first order. Activations and gates change the constant, and correlations accumulated during training invalidate the independence assumptions, so Equation 17 is an initialization heuristic rather than a guarantee. It is nevertheless a useful audit: a projection whose standard deviation ignores fan-in can make scale grow or shrink immediately with width.

6.2 Residual Accumulation with Depth

Residual addition creates a second scale problem. Under the simplifying assumption that the residual branch updates are zero mean and uncorrelated with the incoming stream,

$$\operatorname{Var}(H^L) \approx \operatorname{Var}(H^0) + \sum_{l=1}^{L} \operatorname{Var}(\Delta H^l). \tag{18}$$

If every branch contributes the same fixed variance, the residual-stream variance grows on the order of L. Practical architectures counter this tendency through normalization, fan-aware initialization, and sometimes depth-dependent scaling of residual projections. GPT-2, for example, reports scaling residual-layer weights at initialization by $\frac{1}{\sqrt{N}}$ for N residual layers to account for accumulation with depth [7]. The exact factor depends on what is counted as a residual branch and where it is applied; copying a constant without matching the architecture defeats the purpose of the derivation.

6.3 Interpreting Gradient-Flow Claims

The identity term in Equation 16 supplies a direct gradient contribution, but it does not make optimization automatically stable. The learned Jacobian can still produce large updates, normalization can amplify small-variance inputs, and finite-precision arithmetic, optimizer state, learning rate, and data distribution all interact with the architecture. Conversely, Post-Norm can train successfully when initialization and learning-rate warmup are chosen appropriately.

Xiong et al. analyze gradients at initialization and connect Post-Norm's placement to large output-layer gradients and the usefulness of warmup, while finding better-behaved initial gradients for their Pre-Norm setting [8]. The correct conclusion is conditional: normalization placement changes the gradient paths and therefore the optimization regime. It is not a theorem that every Pre-Norm model is stable or that every Post-Norm model is unstable.

7 Parameter, Compute, and Memory Accounting

The leading costs of the FFN are summarized below. Biases, normalization parameters, elementwise activations, and residual additions are lower order in parameter count relative to the dense projections. The activation-memory column describes logical intermediates; fused kernels, recomputation, and checkpointing can reduce what must be retained.

FFN form	Leading parameters	MACs per token	Width-m intermediates
Vanilla	$2dm$	$2dm$	One projected tensor before contraction
Gated / SwiGLU	$3dm$	$3dm$	Two projections before elementwise gating

Table 2: Leading projection costs for bias-free FFNs. One multiply-accumulate is counted as one MAC; a FLOP convention that counts multiplication and addition separately doubles these values.

For a batch of sequences, multiply the per-token arithmetic by BT. A vanilla FFN therefore performs on the order of $2BTdm$ MACs, while a gated FFN performs $3BTdm$. Normalization and residual addition require only $O(BTd)$ arithmetic, but they read and write full residual-stream tensors and can be limited by memory bandwidth. Operator fusion is valuable because it avoids materializing every normalized vector, bias result, activation, gate, and residual update as a separate memory round trip.

The FFN often owns more parameters than attention within a dense Transformer layer. With $m = 4d$, a vanilla FFN has approximately $8d^2$ weights, compared with roughly $4d^2$ for full-width query, key, value, and output projections in conventional MHA. This comparison concerns projection weights, not total runtime: at long sequence lengths, attention's pairwise position cost can dominate even when its parameter count is smaller.

8 Implementation Contracts

A reliable implementation should make shapes and axes explicit at module boundaries. Given input $B \times T \times d$, an FFN must return exactly $B \times T \times d$; normalization must reduce only the feature axis;

and residual addition must not rely on accidental broadcasting. For gated FFNs, checkpoint conversion must preserve the identity and ordering of the gate and value projections. Swapping them changes Equation 6 because SiLU is applied to only one branch.

Numerical checks should include finite outputs for nearly constant vectors, agreement between training and inference normalization, and a reference comparison in higher precision. Initialization tests can measure empirical output variance for random inputs and verify that every intended residual projection received its depth scaling. These checks do not replace end-to-end training, but they catch silent specification errors before optimizer behavior is blamed for an incorrect tensor program.

9 Summary

The position-wise FFN expands each token representation, applies nonlinear or gated feature interactions, and contracts the result back to the residual width. SwiGLU adds a learned multiplicative gate and therefore requires a narrower intermediate dimension when compared at fixed parameter count. LayerNorm centers and rescales each token vector, whereas RMSNorm rescales by its second raw moment without centering. Residual connections accumulate sublayer updates into a shared stream, and the placement of normalization determines whether the identity path itself passes through a normalizer.

These choices jointly determine more than a block diagram. They set the leading FFN parameter and compute budget, the shape of saved activations, the numerical behavior of reductions, and the routes by which gradients traverse depth. Initialization and residual scaling should therefore be derived from the actual block organization rather than selected as isolated defaults.

References

[1] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

[2] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016, [Online]. Available: https://arxiv.org/abs/1606.08415

[3] N. Shazeer, "GLU Variants Improve Transformer," *arXiv preprint arXiv:2002.05202*, 2020, [Online]. Available: https://arxiv.org/abs/2002.05202

[4] H. Touvron *et al.*, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023, [Online]. Available: https://arxiv.org/abs/2302.13971

[5] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv preprint arXiv:1607.06450*, 2016, [Online]. Available: https://arxiv.org/abs/1607.06450

[6] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019.

[7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language-models.pdf

[8] R. Xiong *et al.*, "On Layer Normalization in the Transformer Architecture," in *Proceedings of the 37th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 10524–10533.